

Problems for Developer and PO

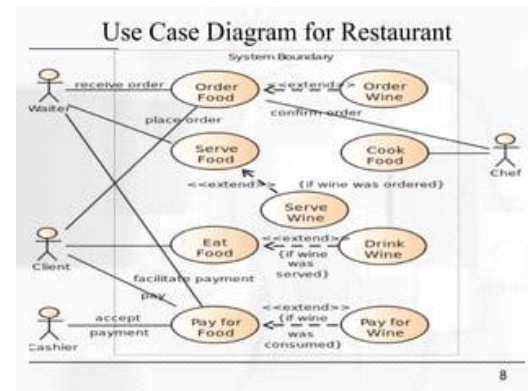
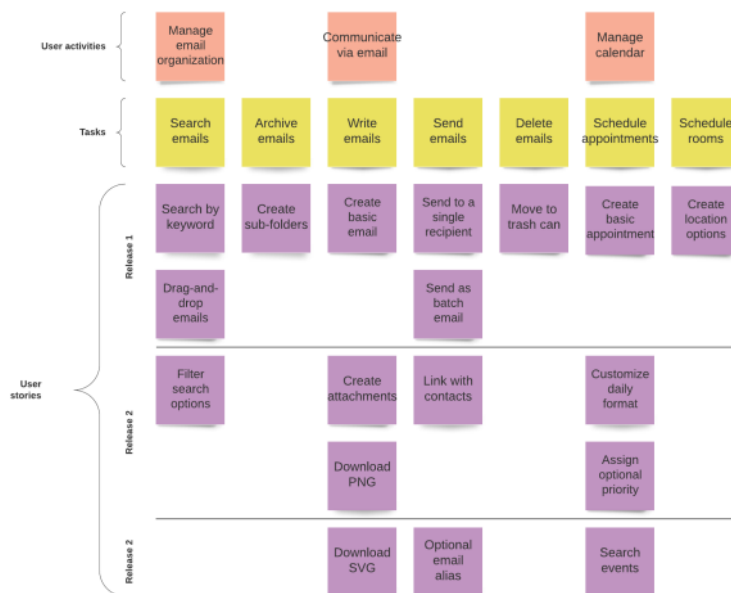
Montag, 9. Dezember 2024 17:53

Goals of Software Development

- Provide more **value** for customer
- All requirements are describing the whole value
- Customer expects
 - Quality
 - Correct behavior
 - Quick changes
- Customer expects not
 - Maintainable code
 - Testable code
 - Only implicitly

Are User Stories / Use Cases good enough for requirements analysis?

- How do User Story Maps or Use Cases organize requirements?



- ? • How to verify **correctness**?
 - Acceptance Tests?
 - But how clear can you describe them if you don't know the code yet that fulfills the criteria?
 - Refer to UI?
- ? • Do increments refer to code?
 - Related to testability?
- **Problem** for developer and also PO:
 - ? ◦ **When** is increment **correct**?
 - Potential of **misunderstandings** and **presumptions**
 - May lead to **unexpected iterations**
- ? • Can requirements be divided recursively into smaller pieces?
 - Maybe to some extent yes
 - ? ◦ But is there **relation to code**?
- User Stories / Use cases of course **still useful, but**
 - **focus** is on **PO** and Product Manager
 - Less focus on developer
 - They do **not touch** / reach the **code** level close enough
- To **solve** this gap of clearness developer needs "a plan"
 - of initially **excluding** a **requirements** area
 - or only building a prototype
-

Slicing - Goals of Requirements Analysis

Dienstag, 16. Juli 2024 15:56

- **Understanding**
- Requirements unclear
 - Huge
 - Complex
 - Confusing
- **Divide and Conquer**
 - Typically known in code (Modularization, Separation of Concerns)
 - Use it also in **requirements**
- ? • But **divide** into **what**?
 - Scopes?
 - Features?
- ? • **Why** divide? Why divide code?
 - Clarity -> Modularization
 - Correctness
 - => **Testability**
- **Slicing** may help as a **systematic** approach to **spot unclarity**
 - Slicing defines clear hierarchy to go through
 - From rough to fine contexts / scopes
 - Scopes on all analysis levels (high and low ones)
 - Each scope and level has a representation in code

When do you know the analysis is finished?

- **Tuple:**
 - Function signature of **Entry Point**: input, name, output
 - **Test cases** per Entry Point function
 - This **tuple** is your **goal**
- Nice to have: Causal chain
 - What steps are necessary to transform the input to the desired output
- „A function with signature <signature> to satisfy the test <test case> by implementing <causal chain>.
- Do not leave PO before you know the tuple
 - Avoids response waiting times and avoids unnecessary local pre optimizations
- If not clear even for PO -> reduce scope or implement prototype
 - Prototype: can be code or other type of analysis e.g. on paper
- At end of discussion it should be clear:
 - **Which function should I implement?**
 - **Trigger function with Input and Output data: triggered by user**
 - **Test cases for each function signature**

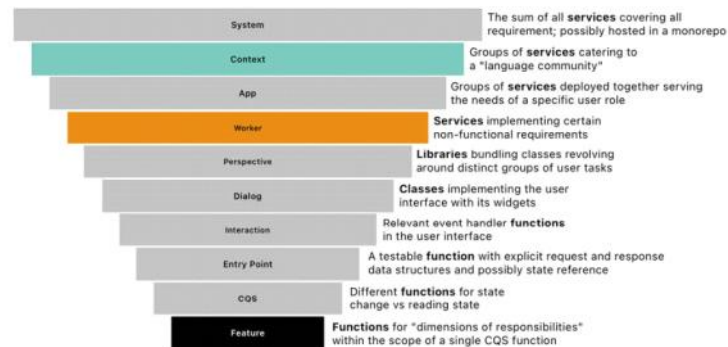
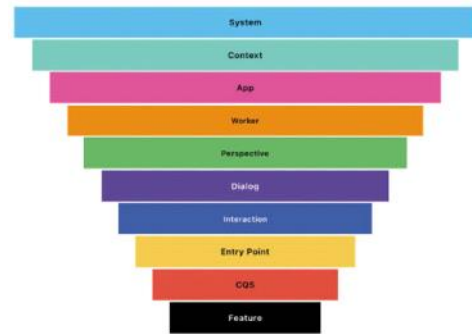
From <<https://www.dotnetpro.de/planung/architektur/strukturiert-zerlegen-2911848.html>>

Hierarchy Levels

Dienstag, 16. Juli 2024 16:28

From <<https://www.dotnetpro.de/planung/clean-code/stier-hoernern-packen-2915202.html>>

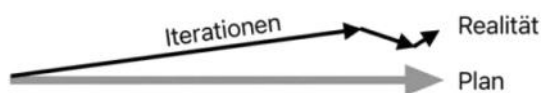
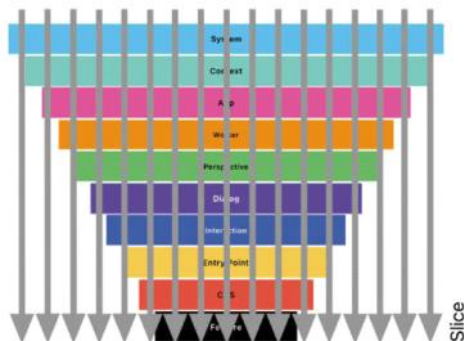
Hierarchy Levels



- **L1 - System:** Das gesamte Softwaresystem in seiner Umwelt im Überblick.
- **L2 - Context:** Das Softwaresystem, zerlegt in große thematische Bereiche. z. B. Amazon: Kunden, Lagerhaltung, Einkauf
- **L3 - App:** Ein Context, zerlegt in unterschiedliche, eigenständige Zugänge für Anwenderrollen. z. B. App für User, App für Admins
- **L4 - Worker:** Eine App, zerlegt in grobe selbstständige Einheiten mit eigenen Schnittstellen für Mensch oder Software. z. B. verschiedene Betriebssystemprozesse für Client und Server
- **L5 - Perspective:** Ein Worker, betrachtet durch unterschiedliche Brillen eines Benutzers. z. B. Amazon: Meine Bestellungen, Shop, Warenkorb
- **L6 - Dialog:** Eine Perspective, zerlegt in thematisch fokussierte, deutlich unterscheidbare „Fenster“ mit ihren Übergängen beziehungsweise Verbindungen.
- **L7 - Interaction:** Die in einem Dialog triggerbaren Funktionen der Software mit ihren Ausgangspunkten und möglicherweise verschiedenen Endpunkten.
- **L8 - Entry Point:** Die „Triggerfunktion“ hinter einer Interaktion mit konkreten Beschreibung der Nachrichten an/von eine(r) „Triggerfunktion“ (Request, Response) und des für sie relevanten Zustands beziehungsweise ihrer Seiteneffekte.
- **L9 - CQS** Klassifizierung eines Entry Points gemäß Command-Query-Separation-Prinzip beziehungsweise Aufspaltung der „Triggerfunktion“ in mehrere Funktionen, die CQS-konform sind.
- **L10 - Feature:** Die Anforderungen an eine „Triggerfunktion“ in feinere Inkremente zerlegen, ausgehend von ihren Nachrichten und ihrem Zustand.
- You may have different structure but there are **two fix points**
 - **System**
 - **Entry Point**
- Some levels (e.g. System) are rarely in focus, some are often (e.g. Interactions)

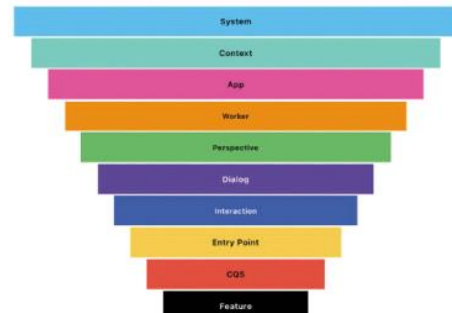
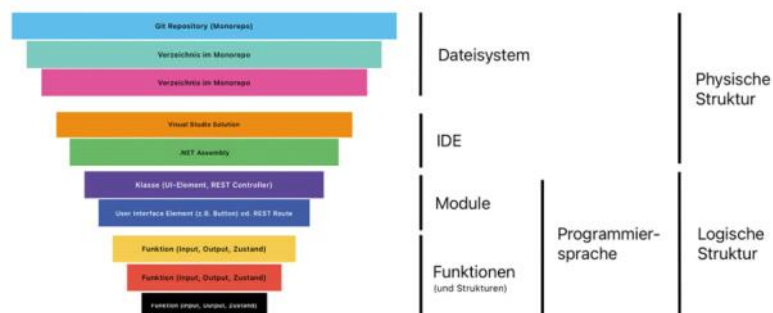
Slicing

- Top down analysis, top down slices (implementations), from outside to inside
- **Thin** slices ->
 - Quick iterations / implementations
 - Quick customer feedback
- A Slice = One Feature of an Entry Point



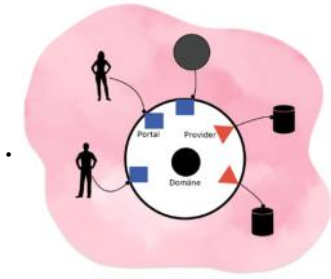
Inkremente können nur iterativ umgesetzt werden; die Ergebnisse weichen am Ende oft von der ersten Idee ab (Bild 6)

Suggestion for a mapping of Slicing hierarchy for developer relevant structures:

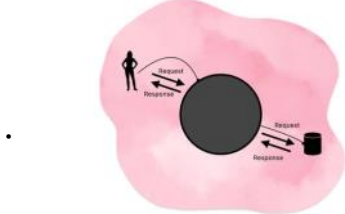


System

- Environment <-> Software communication
- User Roles (types of users, can also be other systems)
 - Portals
- Resources (file system, database, other systems,...)
 - Providers



- Request and Response



Checklist

- All **user roles** known?
- All **states** (internal/external) / **resources** known? Easily overseen e.g. time, random numbers generator
- Do we know the **domain** now?
- Which **technologies** (frontend, ...)?
- Team **know-how** about the technology?
- **Interactions** between system and users/**environment**?
 - Function signature, which state is required and how to be used by function
 - Test cases
 - Which trigger function should I implement?
- **Non-functional** requirements (performance, security...)?
- **Data** structures?

Still Unclear how to get to Entry Point?

- Think from **bottom to top**
- Entry point unclear?
 - What about next level interactions?
- Interactions unclear?
 - What about next level dialogs and their connections?
- Dialogs unclear?
 - What about perspectives of users and how they see the system and their interaction?
- Level clear?
 - YES -> Go DOWN
 - NO -> Go UP

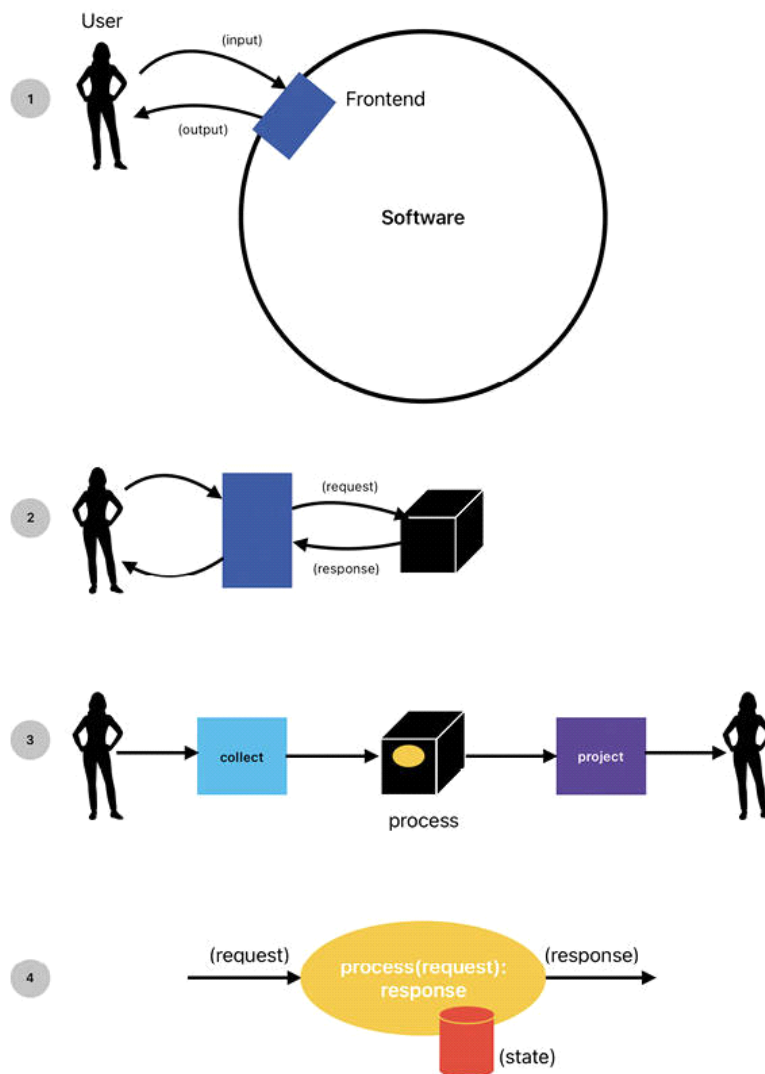
Interactions (Level 7)

Dienstag, 23. Juli 2024 16:36

Users want Interactions

Do not leave PO until Interactions are clear

Basic code structure of processing interactions

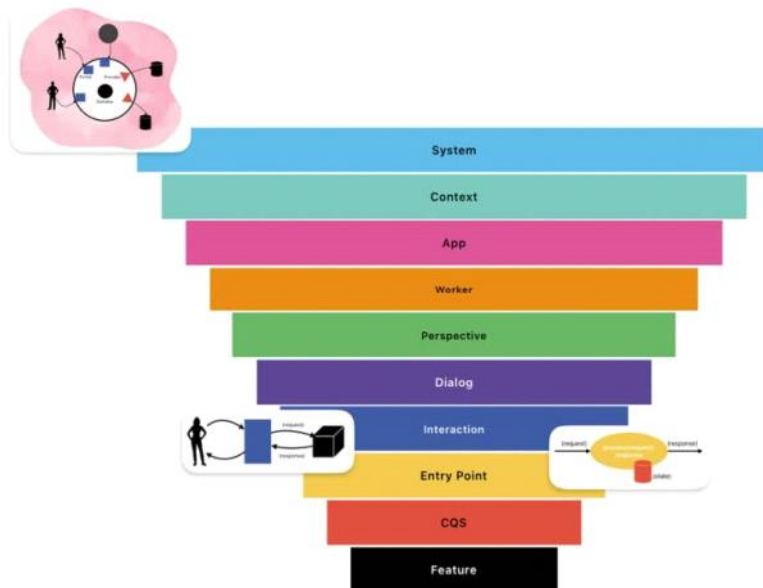


- Collect request input data via UI technology in Portal (Adapter)
- Process = Trigger function = Entry Point
- Project response output data via UI technology in Portal (Adapter)

```
void MyButton_click(...) {  
    var request = Collect(...);  
    var response = blackbox.Process(request);  
    Project(response);  
}
```

Event handler calls Trigger function

- Event handler / Command (WPF) are part of UI technology (within Portal as Adapter)



- Interactions can be **hard to identify**. Dozens/**Hundreds** of **interactions** possible in **complex** systems. Customer cannot answer it in the first place.
- **Interaction is the answer to: What should happen when?**
 Hier sind typische Fragen, mit deren Hilfe Sie das Gespräch leiten können:
 - „Welche Buttons möchtest du klicken können?“
 - „Welche Menüpunkte brauchst du?“
 - „Sollen Tastendrücke bei Eingabe der Daten das Verhalten auslösen?“
 - „Wann und wie soll Verhalten X eigentlich ausgelöst werden?“
 - „Kommt es zu Verhalten Y durch einen ‚Reiz‘ des Anwenders oder automatisch?“
- The more requirements the less a PO can say in early analysis phase. Thus we have hierarchy **levels above interactions**
- Sometimes not so important if interaction is triggered by button or menu.
- More **important to distinguish behavior X** from Y
- **State in frontend?** Trigger function **returns all data** or only **new data**?
 - Another trigger function could do it differently
 - -> UI Technology must be clear

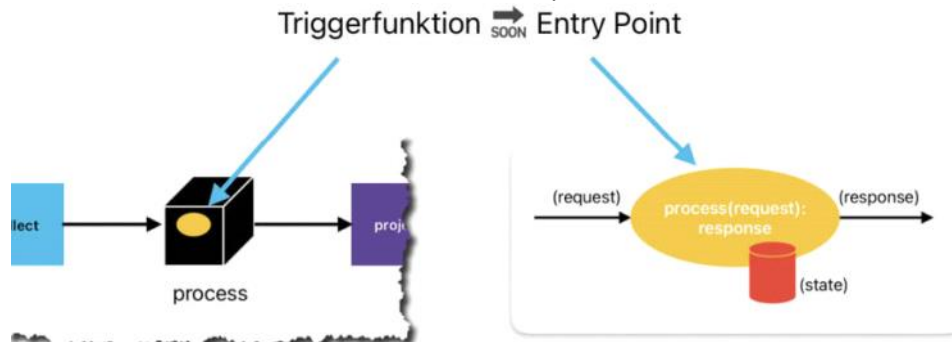
From <<https://www.dotnetpro.de/planung/clean-code/messer-richtig-ansetzen-2918866.html>>

Entry Points (Level 8)

Freitag, 13. September 2024 17:11

- What is the **difference** between **Trigger** function and **Entry Point**

- **Same** function on **different levels** of hierarchy



- Left: Interaction Level

Right: Entry Point Level

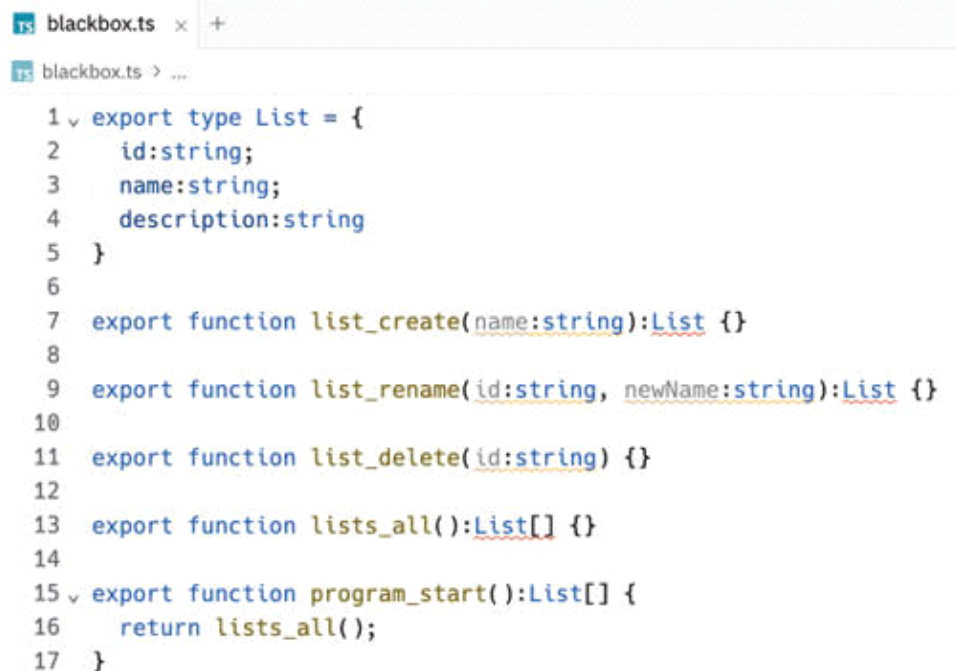
- **Trigger** function during PO discussion on **Interaction** Level
 - Focus on **user** interaction
 - For understanding that there is such a function with its request/response data in general (e.g. HTTP Endpoint / Api Controller)
- **Entry Point** more **concrete**, where the real behavior is implemented (in the backend)
 - Focus on **technical** implementation
 - **Request**/Input data
 - **Response** data
 - May produce **state/side effect**
- **State** within **frontend** or **black box**?
 - **Signature depends** on it
 - **Depends** on **where** and how you can **test more code** more **easily**?
 - If state handling in frontend requires more effort than do it in black box (backend). If trivial in frontend where code does not requiring to be tested then do it in frontend
 - Example: Create Todo List
 - `list_create(name: string):List[]`
 - ◆ No state handling in Frontend required
 - `list_create(name: string):List`
 - ◆ Frontend needs to hold state to show all lists
- When **not considering** the **environment** (e.g. UI technology) and it's **interactions** with the core (black box) of the software system, that could be a **problematic pre optimization**.
 - It is ok that the UI/UX paradigm is considered in the black box but of course the UI Framework functionality must be encapsulated within Portal
 - Must be a conscious decision between efficiency and flexibility (to be able to change UI technology)

Example

- Trigger functions:

- Liste einrichten: *list_create()*
- Liste umbenennen: *list_rename()*
- Liste löschen: *list_delete()*
- Listen anzeigen: *lists_all()*
- Programm starten: *program_start()*

- Entry points:



```

1  export type List = {
2    id:string;
3    name:string;
4    description:string
5  }
6
7  export function list_create(name:string):List {}
8
9  export function list_rename(id:string, newName:string):List {}
10
11 export function list_delete(id:string) {}
12
13 export function lists_all():List[] {}
14
15 export function program_start():List[] {
16   return lists_all();
17 }

```

- Either test each entry point separately or test with scenario test, e.g.:
 - Das Programm wird gestartet; noch gibt es keine Listen.
 - Es wird eine Liste hinzugefügt.
 - Die Listen werden alle abgerufen.
 - Es wird noch eine Liste hinzugefügt.
 - Die Listen werden wieder abgerufen.
 - Eine Liste wird umbenannt.
 - Eine andere Liste wird gelöscht.
 - Die Listen werden wieder abgerufen.

Scenario tests just cover basic workflow (end to end) but not all conditions and logic of the code


```

blackbox_tests x +
blackbox_tests > ...

1 import * as sut from "../blackbox.ts";
2
3 import { assertEquals } from "https://deno.land/std@0.167.0/testing/asserts.ts";
4
5 Deno.test("scenario", () => {
6   // 1. Das Programm wird gestartet; noch gibt es keine Listen.
7   var lists = sut.program_start();
8   assertEquals(lists.length, 0);
9
10  // 2. Es wird eine Liste hinzugefügt.
11  var list = sut.list_create("L1");
12  assertEquals(list.name, "L1");
13  assertEquals(list.description, "");
14  const lid1 = list.id;
15
16  // 3. Die Listen werden alle abgerufen.
17  lists = sut.lists_all();
18  assertEquals(lists, [
19    {
20      id: lid1,
21      name: "L1",
22      description: "",
23    },
24  ]);
25
26  // 4. Es wird noch eine Liste hinzugefügt.
27  list = sut.list_create("L2");
28  const lid2 = list.id;
29
30  // 5. Die Listen werden wieder abgerufen.
31  lists = sut.lists_all();
32  assertEquals(lists, [
33    {
34      id: lid1,
35      name: "L1",
36      description: "",
37    },
38    {
39      id: lid2,
40      name: "L2",
41      description: "",
42    },
43  ]);
44
45  // 6. Eine Liste wird umbenannt.
46  list = sut.list_rename(lid1, "Liv2");
47  assertEquals(list.id, lid1);
48  assertEquals(list.name, "Liv2");
49
50  // 7. Eine andere Liste wird gelöscht.
51  sut.list_delete(lid2);
52
53  // 8. Die Listen werden wieder abgerufen.
54  lists = sut.lists_all();
55  assertEquals(lists, [
56    {
57      id: lid1,
58      name: "Liv2",
59      description: "",
60    },
61  ]);
62 });
63

```

- In simple cases you have a few **simple interactions** and you can go **breadth-first**
- or concentrate on **single interaction (depth-first)**
- It **depends how breadth** you can work
- Important to have **at least one entry point** to complete and then proceed with next

Command Query Separation - CQS (Level 9)

Montag, 21. Oktober 2024 17:21

CQS means Slicing on method level

- Separation of Command and Query
- Command
 - **Change** state
 - No return value except metadata like ID of new dataset or count of changed/deleted datasets
 - Can be seen like a **POST** in web
- Query
 - Just **read** / return values
 - No state change
 - **GET** in web
- Advantages
 - **Separate testability** of these different approaches and the **separate handling** of data
 - Technological separation of these modes; for example:
 - **Commands** may require **locks**
 - but **Queries** do **not**
 - Reusability in different contexts due to more fine-grained functions.
- From this perspective, entry points are often unclear. They offer potential for refinement by narrowing the scope to focus on command versus query.

Example

```
1 export type List = {
2   id:string;
3   name:string;
4   description:string
5 }
6
7 const _lists:List[] = [];
8
9 > export function list_create(name:string):List { ... }
18
19 > export function list_rename(id:string, newName:string):List { ... }
26
27 > export function list_delete(id: string) { ... }
33
34 > export function lists_all():List[] { ... }
37
38 > export function program_start():List[] { ... }
```

- Command
 - list_delete
- Query
 - program_start
 - lists_all
- Mixed
 - list_create
 - list_rename
 - Method names **seem** to be **commands** but they return data
 - Important to **recognize** this and make a **conscious decision** to split or not to split e. g. when
 - the data to be **returned** is **"generated"** naturally during command execution and data is not too much
 - **performance** is critical

Refactored list_rename:

```
19 export function list_rename(id:string, newName:string):List {
20   const list = _lists.find((l) => l.id === id);
21   if (list === undefined) throw new Error("Rename: List not found!");
22
23   list.name = newName;
24   return list;
25 }
```

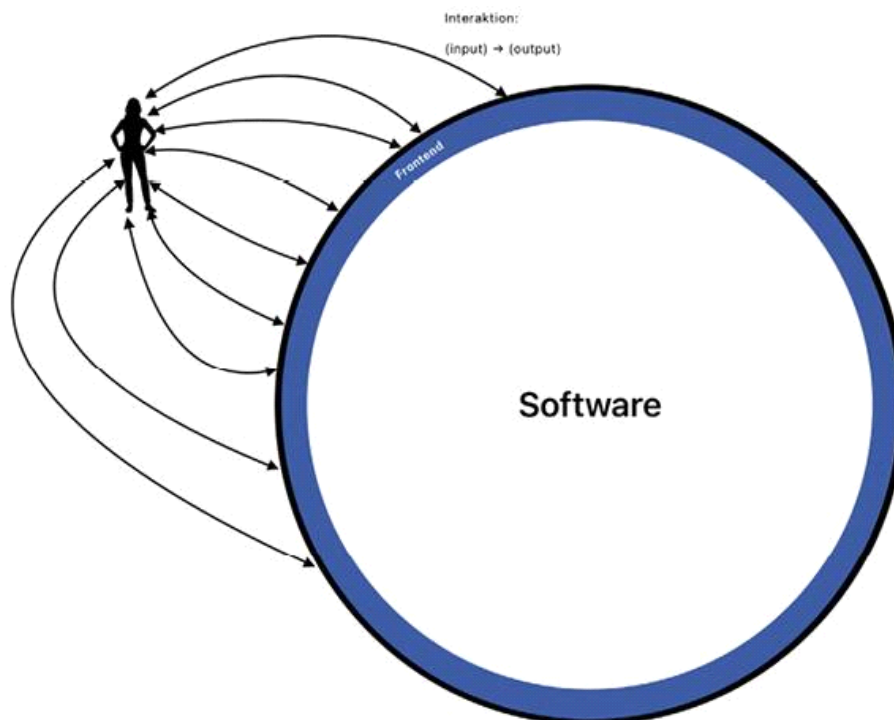


```
21 export function list_rename(id:string, newName:string) {
22   const list = _list_get(id).list;
23   list.name = newName;
24 }
```

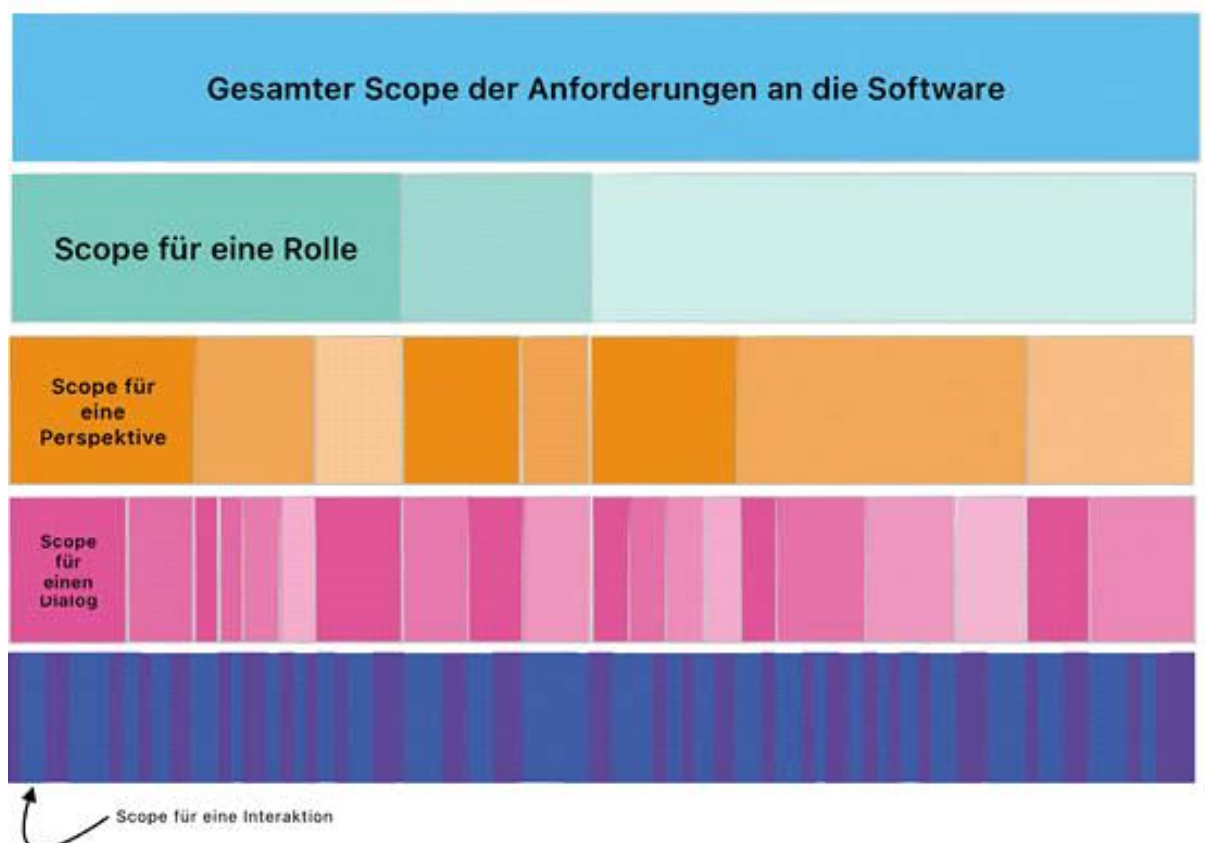
UI drives the Slicing

Mittwoch, 23. Oktober 2024 23:31

- Problem in requirements analysis: Hundreds of Entry points possible
 - PO cannot give the Entry points, even Interactions are not all known

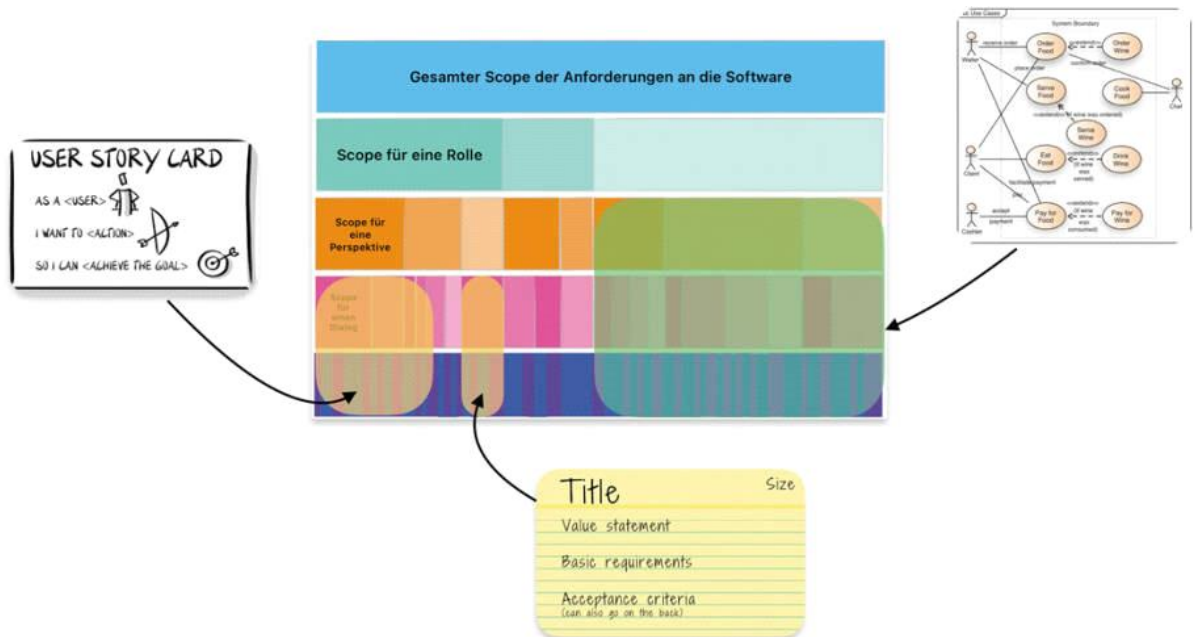


- Hierarchy of Scopes

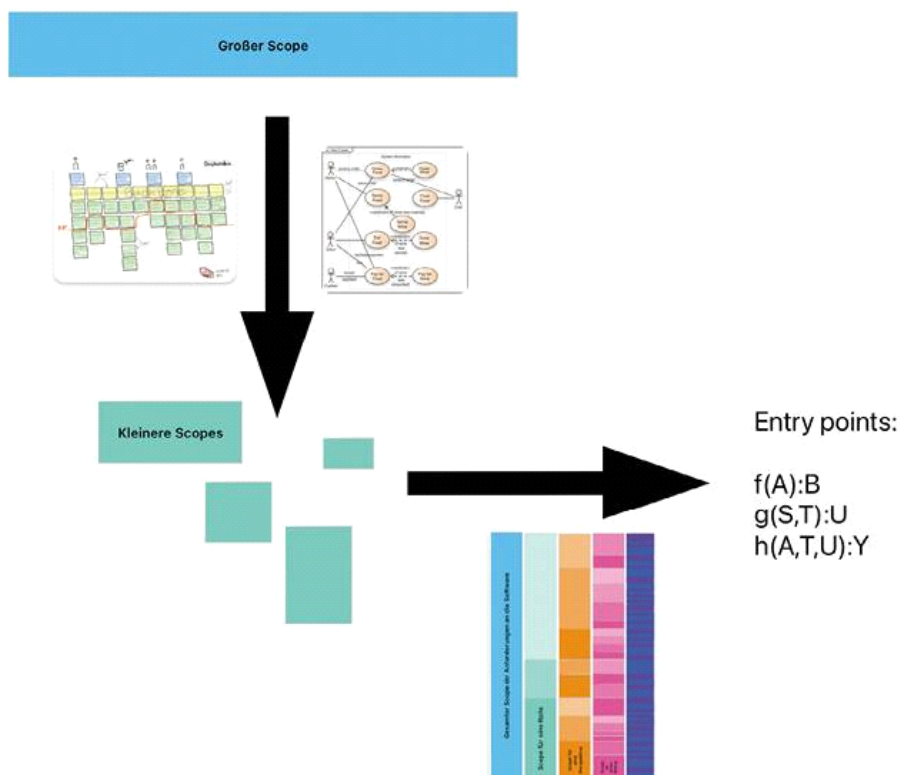


Slicing hierarchy orthogonal to User Stories / Use Cases

- User Stories / Use cases
 - Focus on user perspectives
 - Rough descriptions of user interactions / software behavior
- Slicing
 - Focus on **developer** and technical **implementation**
 - **Detailed breakdown** of requirements
 - Provides concrete **technical artifacts / structure elements** like Entry Points, CQS Functions
- User Stories / Use Cases define scopes that have many slices in slicing hierarchy
 - Developer has no starting point for design and coding



- Developer needs to slice to find Entry points to have a starting point for design and coding



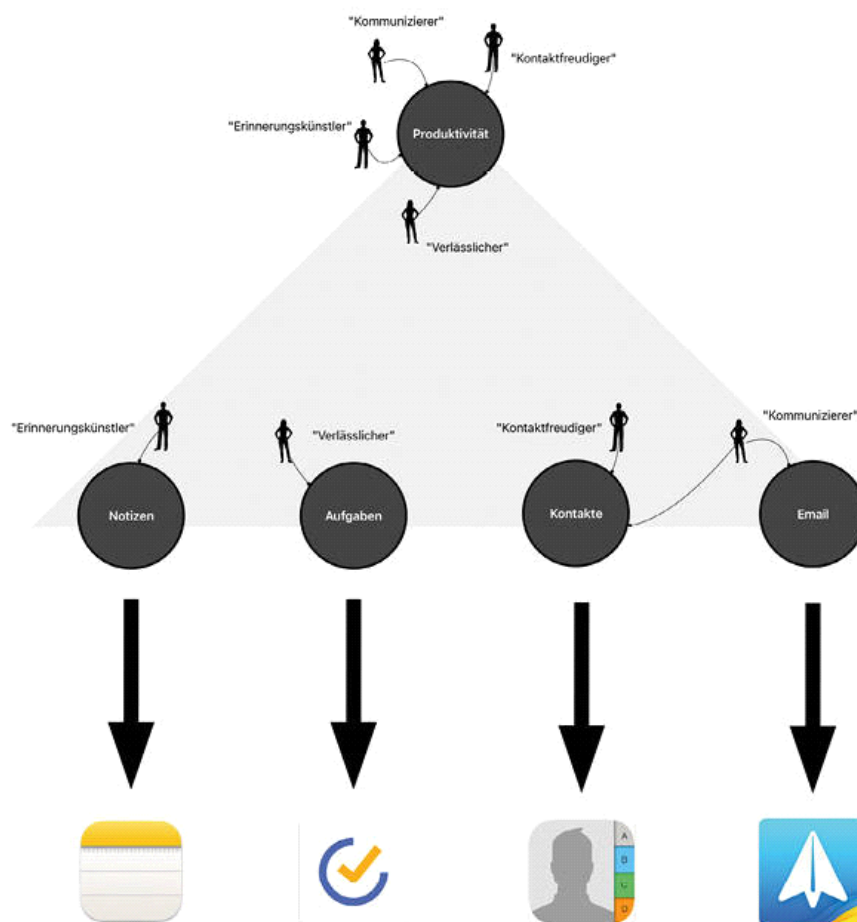
Applications

- First look at roles that user of system has
- ? • For each role separate app or all in one app?

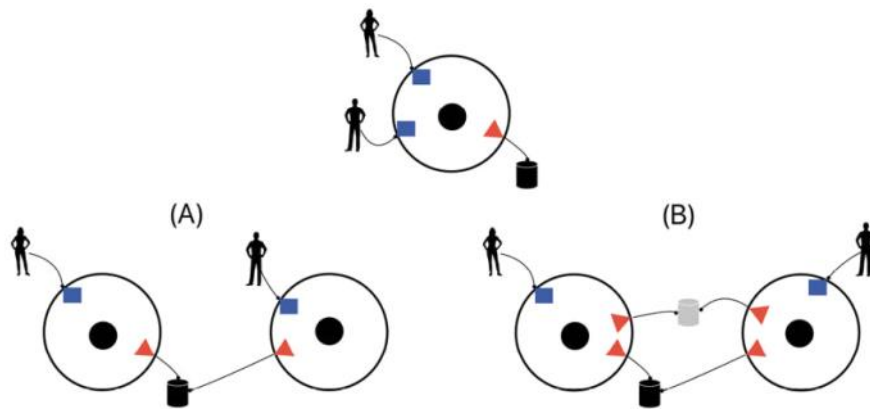
- One size fits all
 - Outlook: Mail, Calendar, Contacts, Tasks



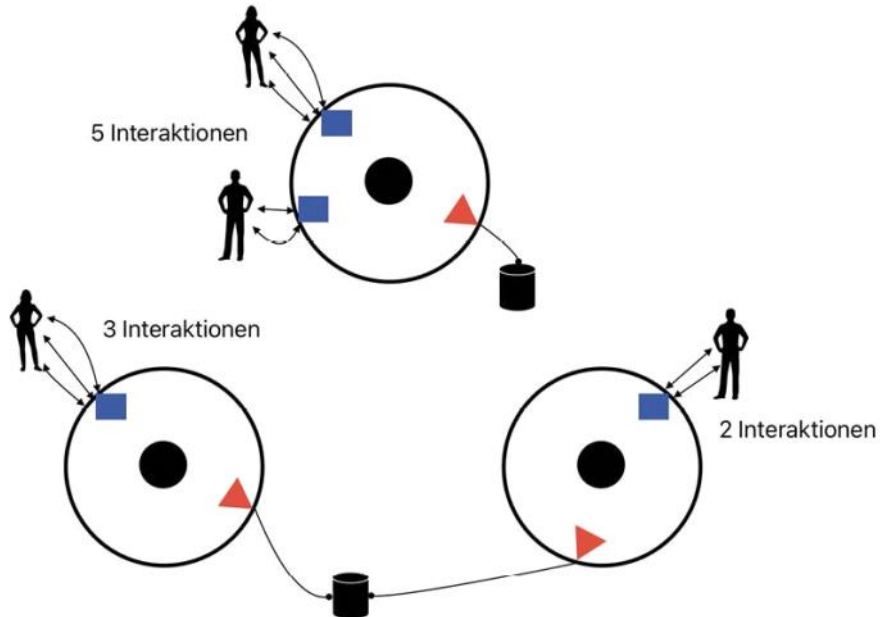
- Best of breed
 - E.g. Smartphone apps with focus on one thing



- Separate Desktop Apps / Icon
- Mix of UI technologies (web, desktop) or platforms possible
- Best possible user experience
- Loosely coupled requirements / implementation
- Parallel work can be easier
- Different Release Cycles
- Disadvantage of effort for connecting apps but can be seen as advantage for loose coupling
- Resources connect apps. Either resource required anyway or extra because of the need to connect apps:

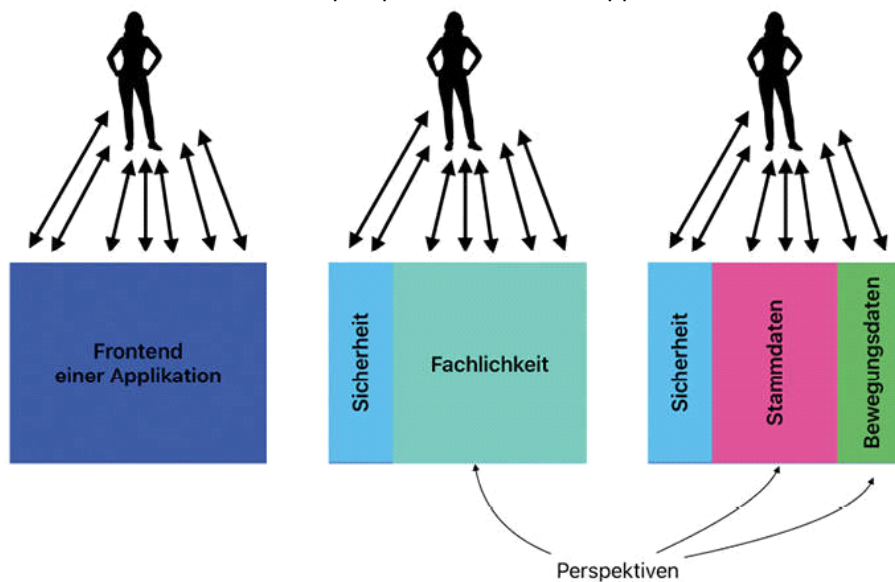


- Less interactions than in one size fits all software:



Perspectives

- ? ○ What if even Applications do not show interactions?
- A user role can have different perspectives within an application



- Do not look to frontend as a monolith
- Divide frontend into more smaller pieces by perspective
 - E.g. Menu with different parts

- Microsoft Word
 - Write Text
 - Create Smart Art / Diagrams
 - Etc.
- Slicing is not about the HOW (UX / Visualization),
- it's about the WHAT and finding
 - interactions / trigger functions that the user wants to do

Dialogs

- Below Perspectives
- One Perspective contains 1 or N dialogs
- Example: Billing system with the following Perspectives and Dialogs

Sicherheit

- Login
- Passwort zurücksetzen

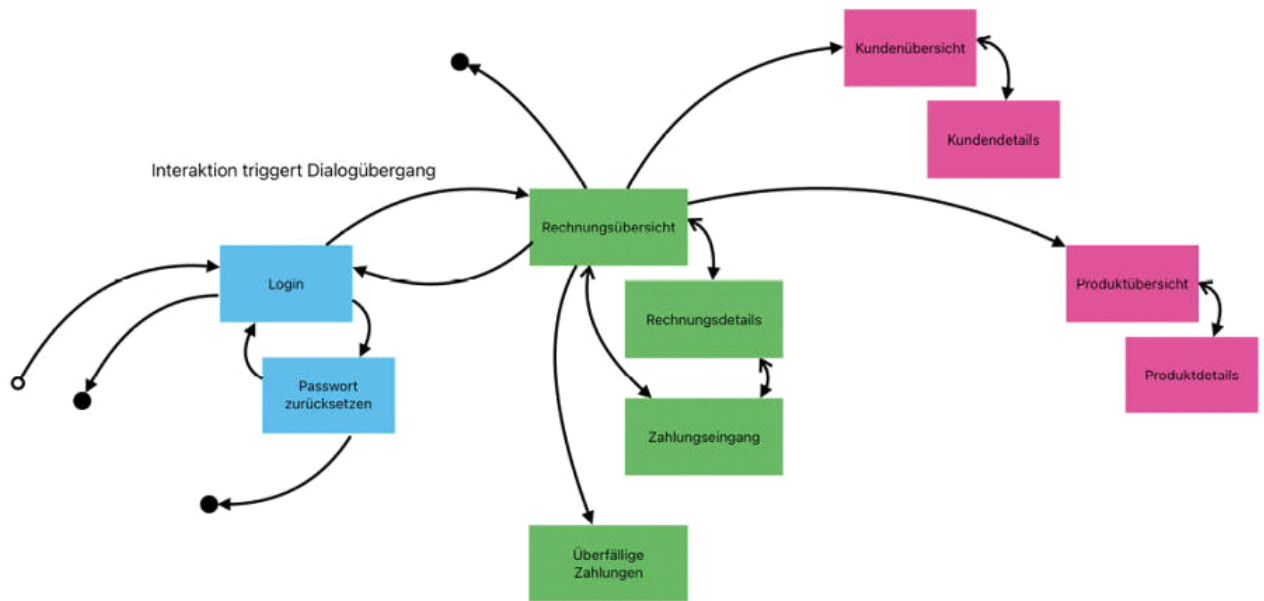
Stammdaten

- Kundenübersicht
- Kundendetails
- Produktübersicht
- Produktdetails

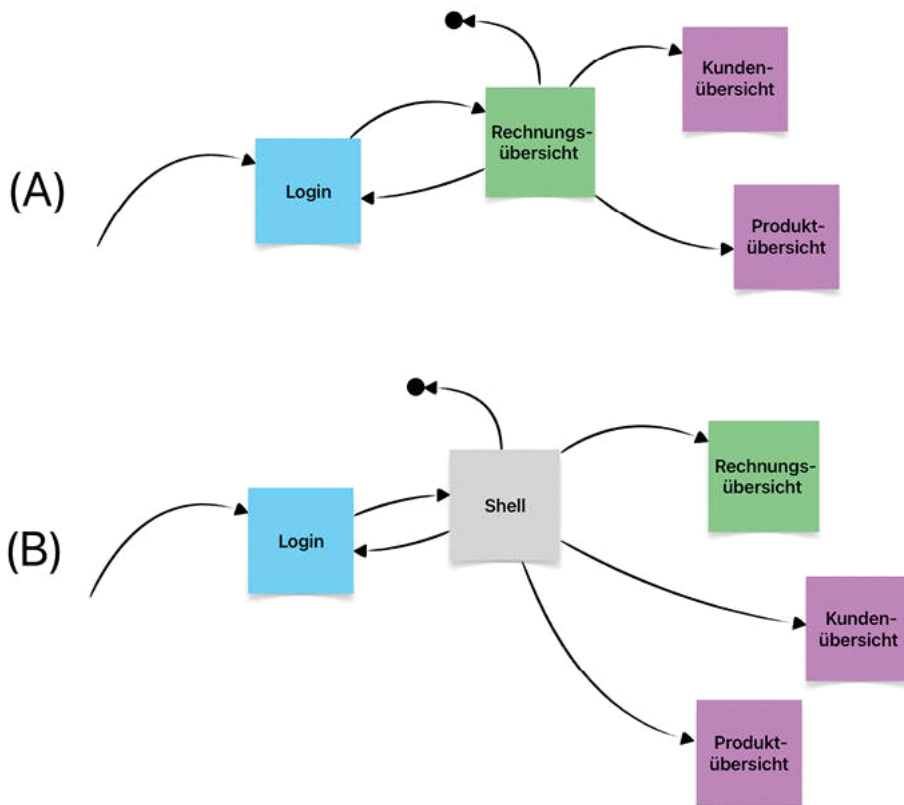
Bewegungsdaten

- Rechnungsübersicht
- Rechnungsdetails
- Überfällige Zahlungen
- Zahlungseingang

- Each Perspective helps to identify the Dialogs
- UI/UX Expert helps for defining Dialogs/UI
- **Which** Dialogs exist
- How do they **depend on** each **other**
- How can user **navigate** from one to another one
 - Each **transition** is an **Interaction**



- Unidirectional
 - Rechnungsübersicht -> Überfällige Zahlungen
 - User can go back but without triggering new behavior, just close dialog
- Forward and Backward
 - Login -> Rechnungsübersicht
 - Maybe back to Login-Dialog for logout
- Sometimes Dialog just occurs
 - Login Dialog during start
- Sometimes no switch
 - Login Dialog -> Reset Password
- Bidirectional
 - Customer overview <-> Customer details
 - Way back not optional
 - Data may have changed and saved
- Post-It notes may help to move and get a feeling for the best navigation between Dialogs
- Could also be a Shell UI (navigation bar e.g. like in smartphone apps)



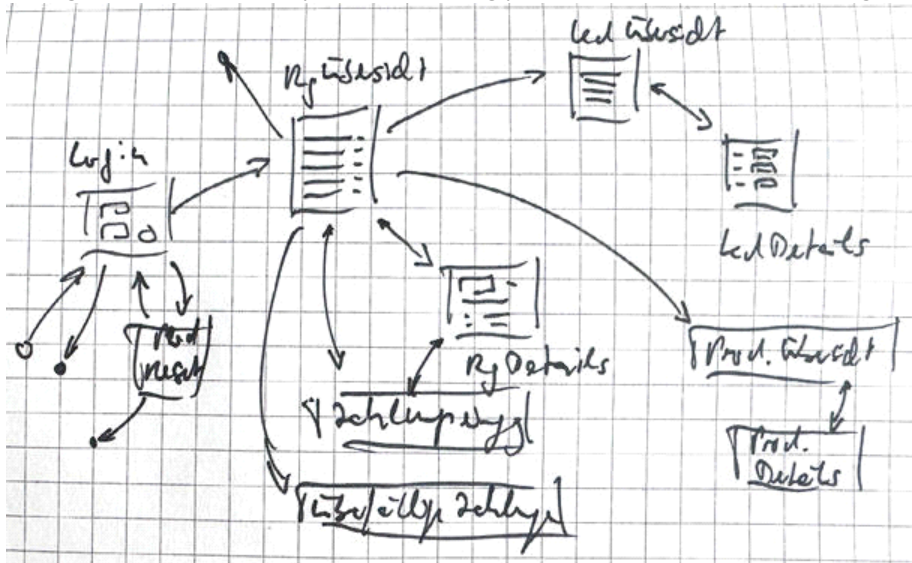
- When you work with **User Stories** and **Use Cases** you need to **discuss Perspectives and Dialogs** with PO

Interactions

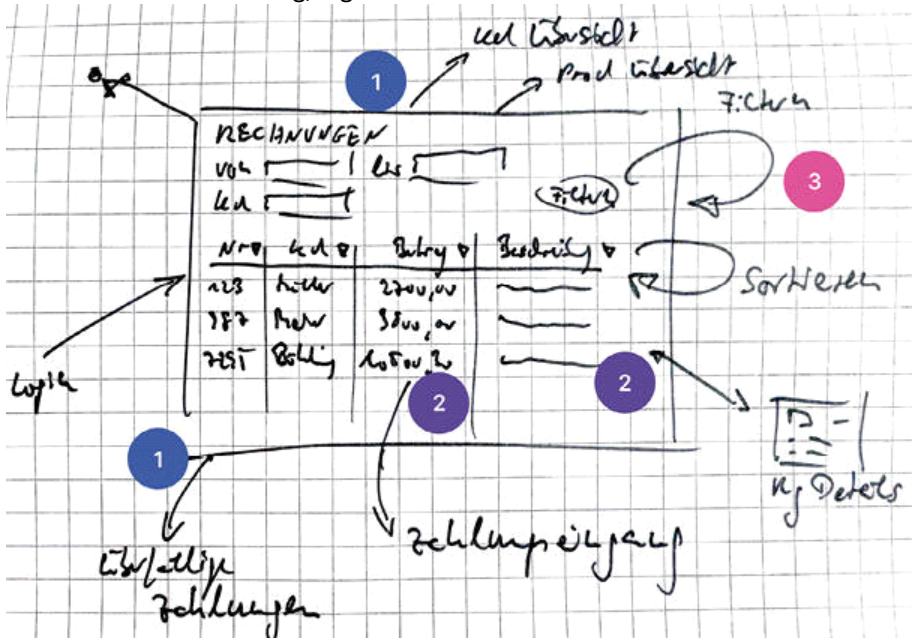
- When you have **identified** the software's **dialogs** or a subset of them that are sufficient **for one increment** then you **know** the **first potential interactions**
- Those who have **transitions** between the dialogs

Next step zoom into dialogs

- ? ○ How does the dialog look like?
- ? ○ What should happen when within the dialog? E.g. in Invoice Overview
 - Create new invoice
 - Display existing invoice
 - Filter
 - Delete
- ? ○ How is the recording of input structured for each interaction?
- ? ○ How is the presentation of the output structured for each interaction?
- ? ○ How exactly should the interaction be triggered? Click, Keyboard etc.
 - => Layout questions => support from UI/UX expert
- ? ○ What Entry Points to call from interactions?
 - Depends on UI technology, e.g. grid control supports filtering then no Entry Point necessary
- Dialogs overview: Use simple **sketch** for big picture **without details** of dialogs and interactions



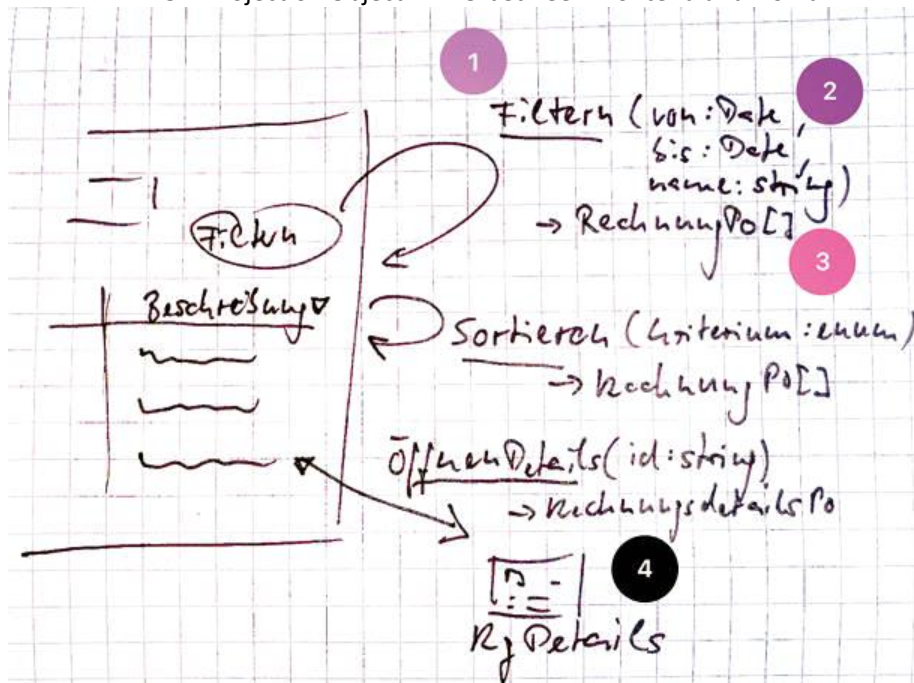
- Pick and sketch one dialog, e.g. Invoice overview



- Some interactions just taken from previous sketch "Dialogs overview" (1) and (2)
- Some arrows are starting from different points
 - Edge: transition to other dialog (1)
 - Within: interaction of specific element (2)
- Some interactions are new, not visible in previous "Dialog overview" e.g. Filter (3)

Entry Points

- From previous Dialogs and it's interactions you know the data and how to display it
- Now possible to define the interface / Entry Point signature
 - Name of action / Entry Point function(1)
 - Filter
 - Parameters of function (2)
 - from, to, date
 - Return value (3)
 - InvoicePO[]
 - PO = Projection Object = DTO between Frontend and Domain



Recap

- Start on System level
 - Try to find user interactions
 - Typically not all interactions are visible here
- Go to next levels and divide previous level into smaller parts / scopes
- Each part of each level is part of an increment / slice
- Each increment adds value for customer
- No level is a must
 - but at least go to System level to see environment
 - Go to the level where you need clarity
 - Try first on interaction level
 - If unclear, see if you can split into Applications, or look for Perspectives in frontend
- At the **end** you should have a result of
 - **Dialogs** with their **interactions**
 - **Entry Point** functions with **tests**
- These should also be the result of PO discussions in case you have User Stories / Use Cases

Context and Worker (Level 2/4)

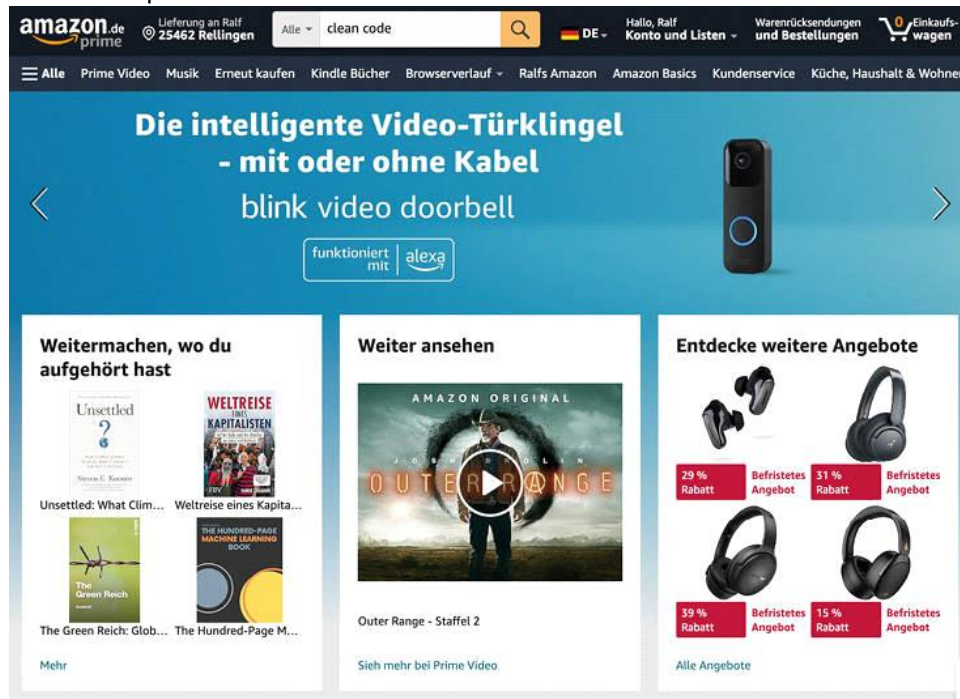
Sonntag, 8. Dezember 2024 13:46

Rough Slices of the Requirements in Big Software Systems

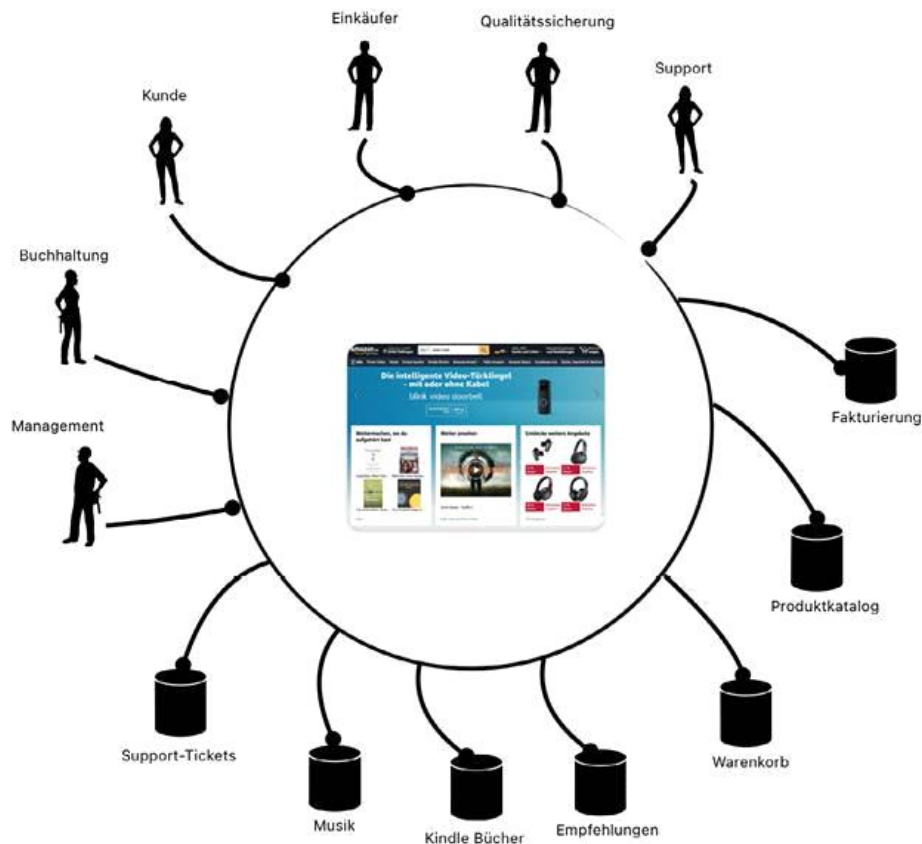
- **Context** and **Worker** useful in **big software** systems

Context

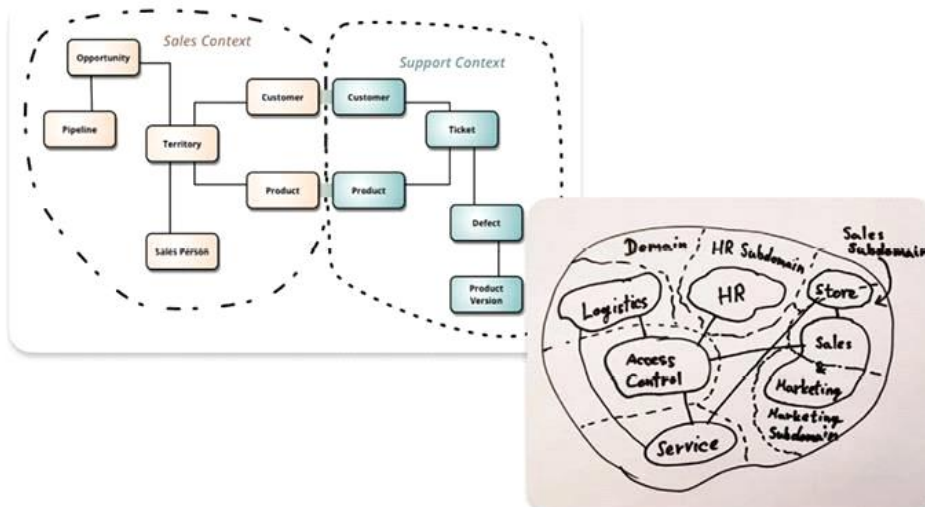
- Example:



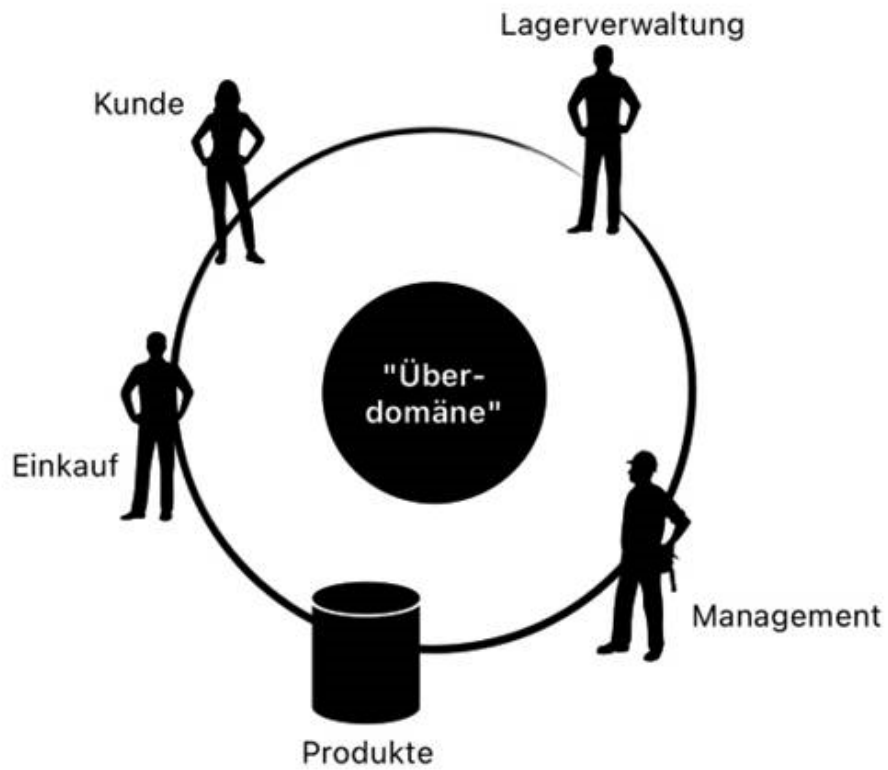
- All Apps known?
- Some may obvious: Web Shop, Prime Video Desktop App
- Even with a rough overview not all apps known:



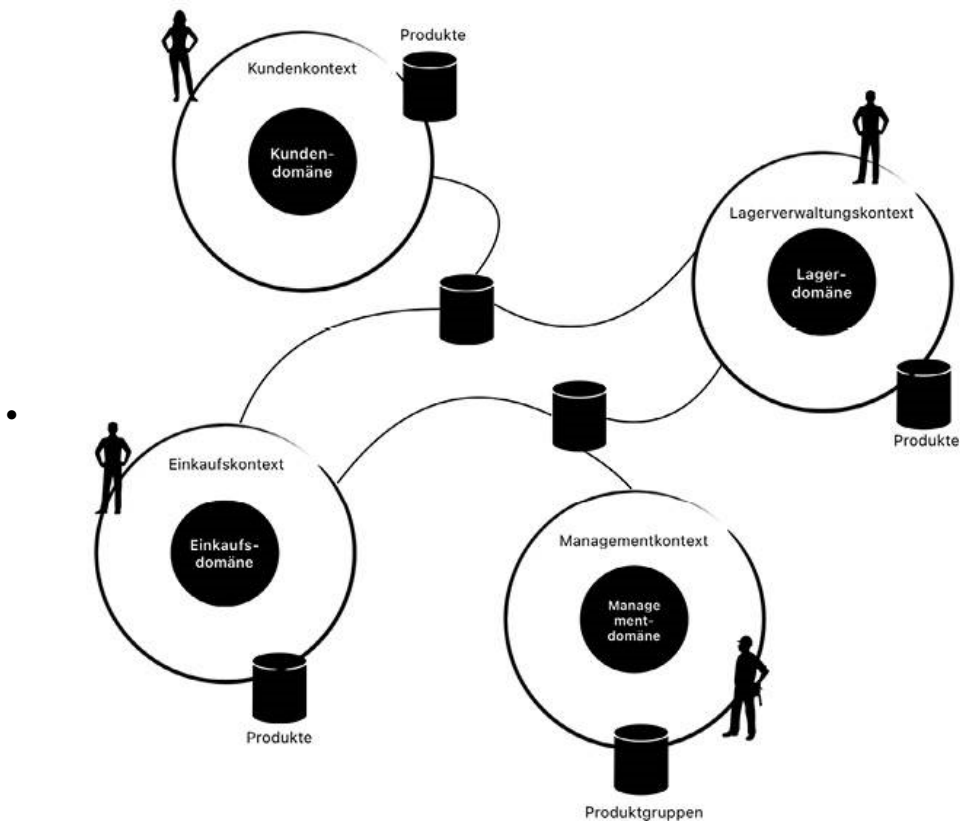
- You cannot see all user roles and resources at this high level
- Need to divide System into Contexts
 - Bounded Contexts in DDD



- ? • How to find Contexts?
 - On App level: you look for single user roles
 - On Context level: Look for **groups of user roles**
 - Roles with **common (domain) language** belong together
- But **problem**
 - e.g. the **word "Product"** is **not** the **same** for Customer as for the Warehouse team
 - Exclusion is needed otherwise you get
 - One size fits all domain
 - One big data model: not flexible
 - Changes are difficult and may produce conflict for other roles



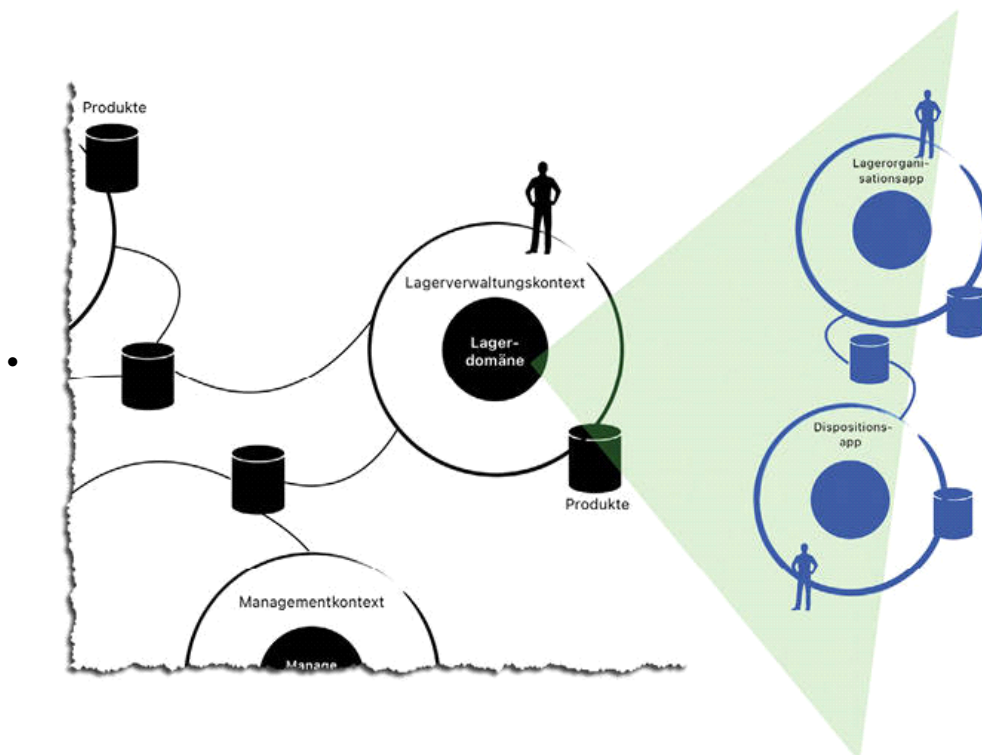
- Context helps to separate with own
 - Domain / language
 - Resources
 - Data models / Persistence
- Each Context is an own subsystem



- Now "Product" exists in each Context
- **DRY** principle is **broken** here by **intention** otherwise
 - Creates dependencies
 - Inflexible
 - Unstable
- Eventual Consistency
 - Is the price to pay for independency, flexibility, clearness
 - No ACID transactions from one context to another
- Contracts between Contexts
 - Platform neutral
 - Maybe different programming languages
- Introduction of Contexts depend on size of the software system
- Be sensible when **terms** are **unclear** and **not unique** for you
 - -> Context required
 - -> Search for **different domain languages**

Worker

- On each Slicing Level you try to find the next smaller parts of elements of current level
- Context (domain languages) -> Apps (single roles) -> ?
 - e.g. Different Apps for warehouse workers and dispatchers



- Typically App is monolith
 - **One process** of OS
 - Simple
 - But code with high coupling if not modularized very well
- But **not all** kind of **requirements** possible with **one process**
 - Security
 - Scalability
 - Robust
- One App can consist of two processes
 - e.g. Client and Server
- Process = Worker
 - Service
 - Microservice
- Requires **effort** for thinking of **communication** architecture
- **Complex**
 - Testing
 - Deployment
 - Operation
 - Expensive
- Only required for **non-functional** requirements
- Must be **sure** you **need it**
- History
 - 40 years: Monolith
 - 30 years: Client / Server
 - 20 years: service oriented (multiple servers)
 - 10 years: Micro services (docker)
 - Recent years: Modulith (component oriented monolith)

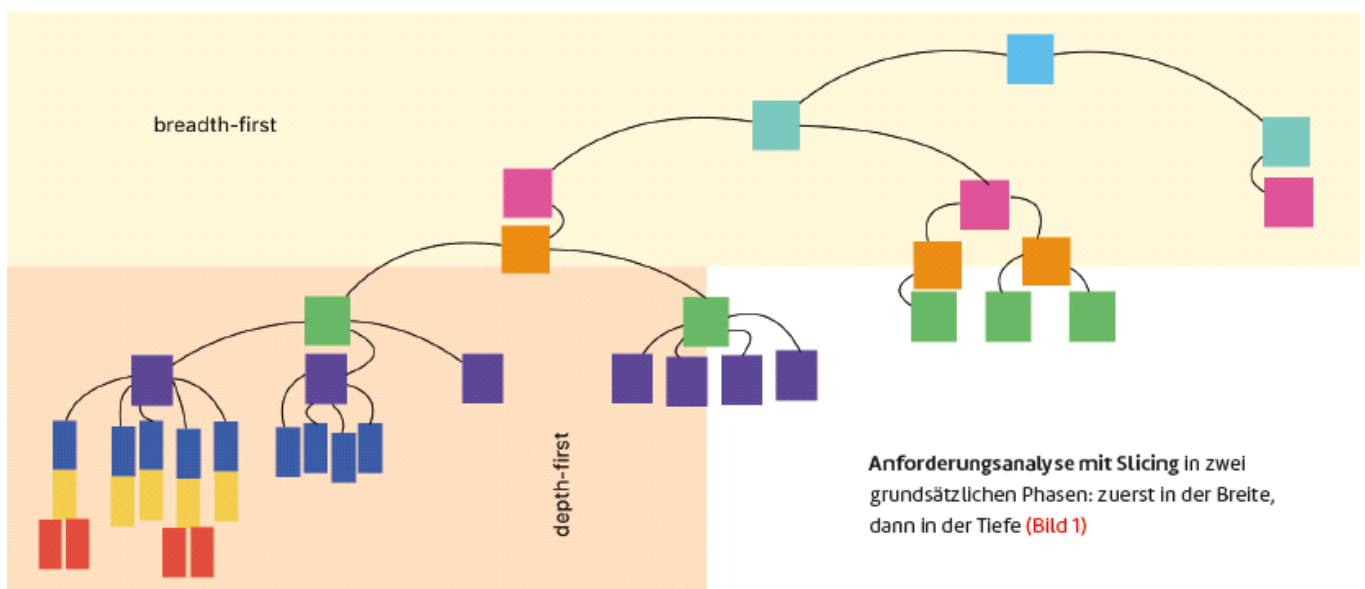
Find Increments within Interactions - Features (Level 10)

Mittwoch, 11. Dezember 2024 17:39

- Slicing helps to
 - Break down requirements in small increments
 - Enable quick feedback
 - Agile software development
- Addresses the questions
 - "whether or not" increment has to be realized
 - Quality of the increment

Existential Increments

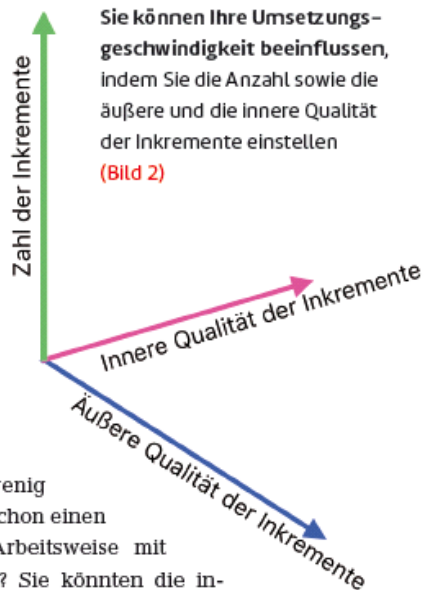
- Focus whether increment **exists or not**
- Entry Points (Functions with signature and test cases)
 - as concrete **starting points**
 - **Lowest** level of **breakdown** of existential increments
- Identification of Entry Points is complete when their existence, signatures and test cases clear
- **Goal:** Find out quickly if requirements **heading in right** direction



- Breadth first
 - System
 - Context
 - App
 - Worker
- Depth
 - At some point pick one branch to go into depth within Worker
 - Perspectives
 - Dialogs
 - Interactions
 - Entry Points
 - CQS

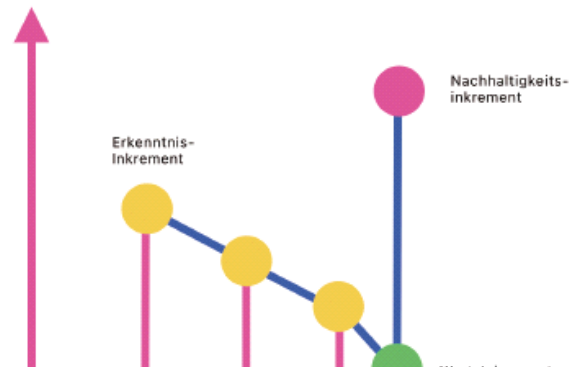
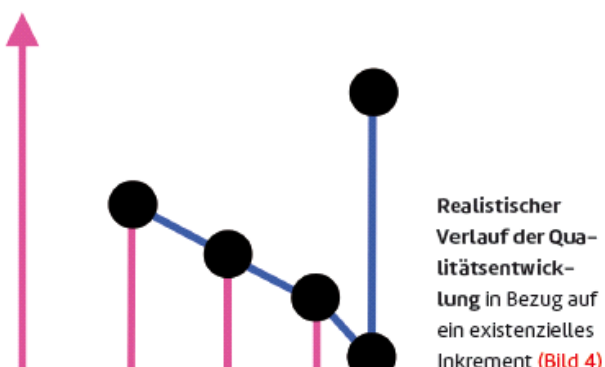
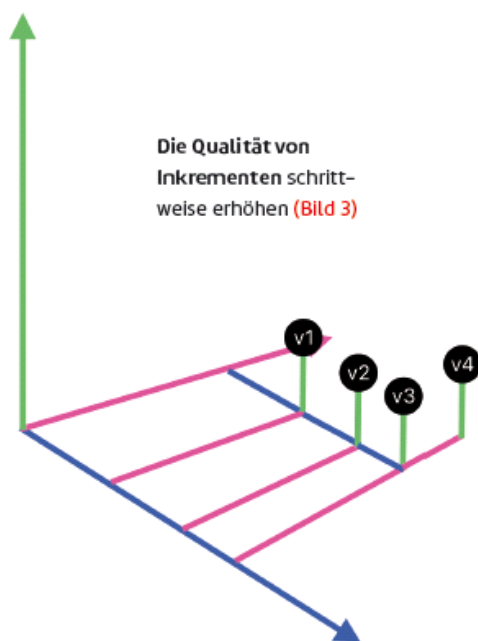
Qualitative Increments

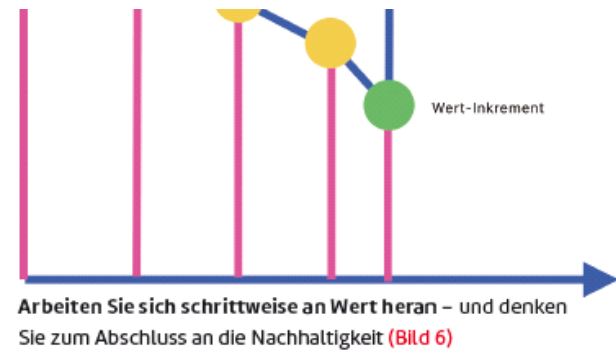
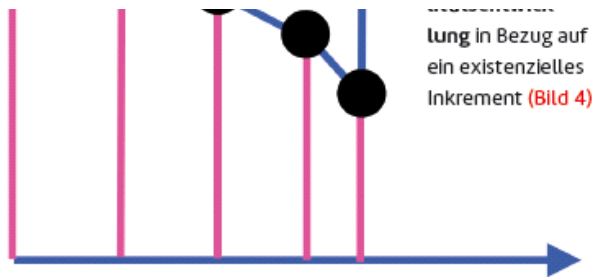
- Describing various **stages of behavior** of **Entry Point implementation**
- Relation to **quality**
- Two dimensions
 - Tangible quality: Directly perceivable by customer/PO e.g. Data persistence in task management example
 - Intangible quality: Internal code quality (testability, modularization)



ie
1 wenig
3 schon einen
r Arbeitsweise mit
ien? Sie könnten die in-

- The **speed** of implementation can be **influenced** by adjusting the **number**, external, and internal **quality** of the increments.
- It is recommended to **initially** implement **few existential** increments with **lower** external and internal **quality** and then **gradually** **improve** them.
- **Value increments** are **deliverable** and generate **added value** for the customer.
- **Insight increments**
 - serve internal **knowledge gain** and help determine if development is on the **right track**.
 - should be strategically placed at the **beginning to learn quickly** and avoid costly detours.





16-Hour Rule and Features

- **Not more recommended** to implement an increment to enable **quick feedback**.
- **Features** are increments **within an EP** and **represent** implemented **decisions** within the **scope of an EP**
- They should be encapsulated in **functions** and **labeled** with an identifier.
- The development of a **value increment** is seen as the **completion of a path of insight increments**

Single Responsibility Principle (SRP)

- The SRP serves as a guideline for analysis and design.
- Each customer decision should be made explicit and represented in a separate unit of code (function, class, module).
- The SRP helps identify features and structure the code so that it can easily adapt to changes.

